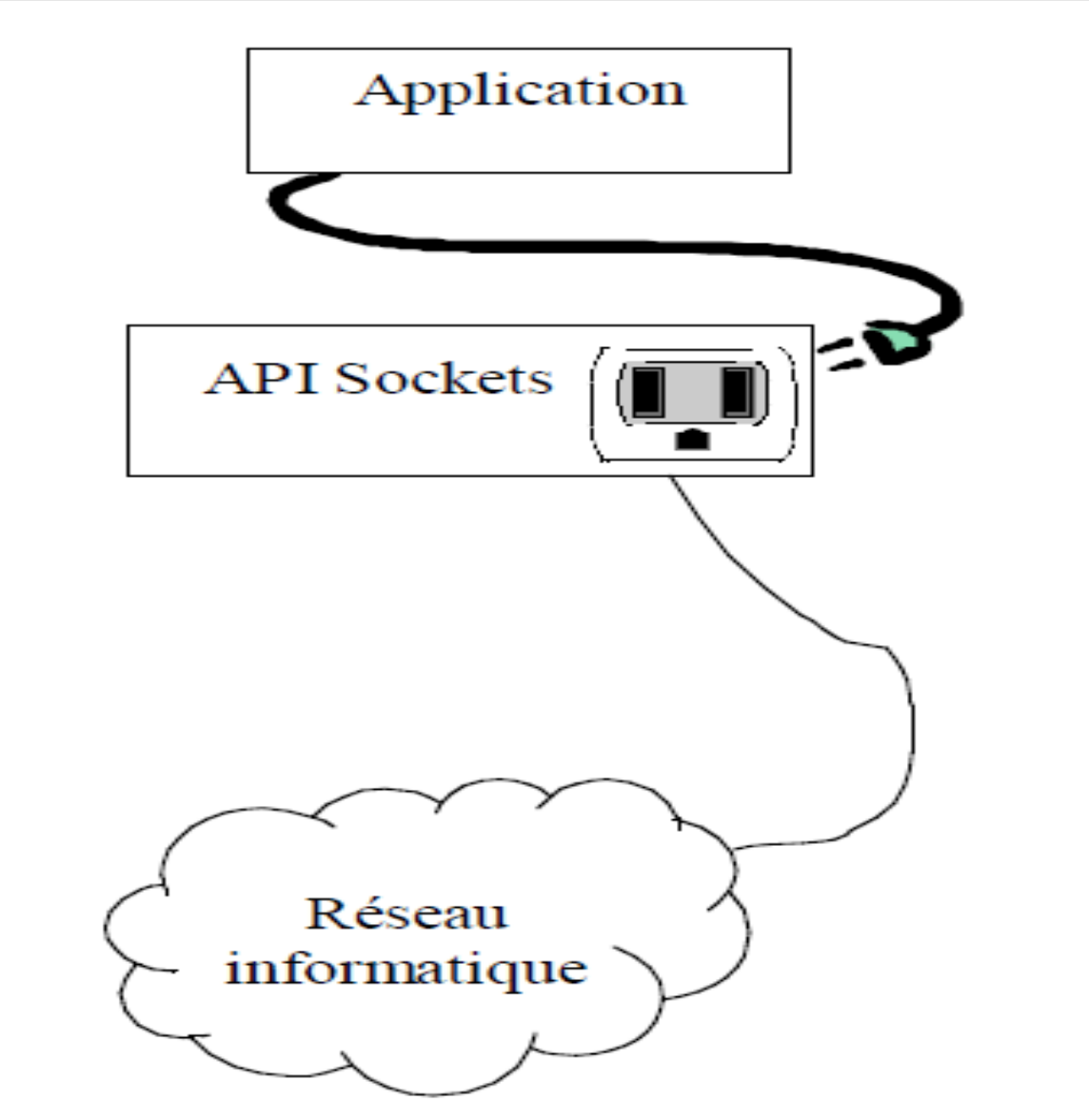
Programmation réseaux

Master 1 CSIA

II-Communications basées sur les Sockets TCP

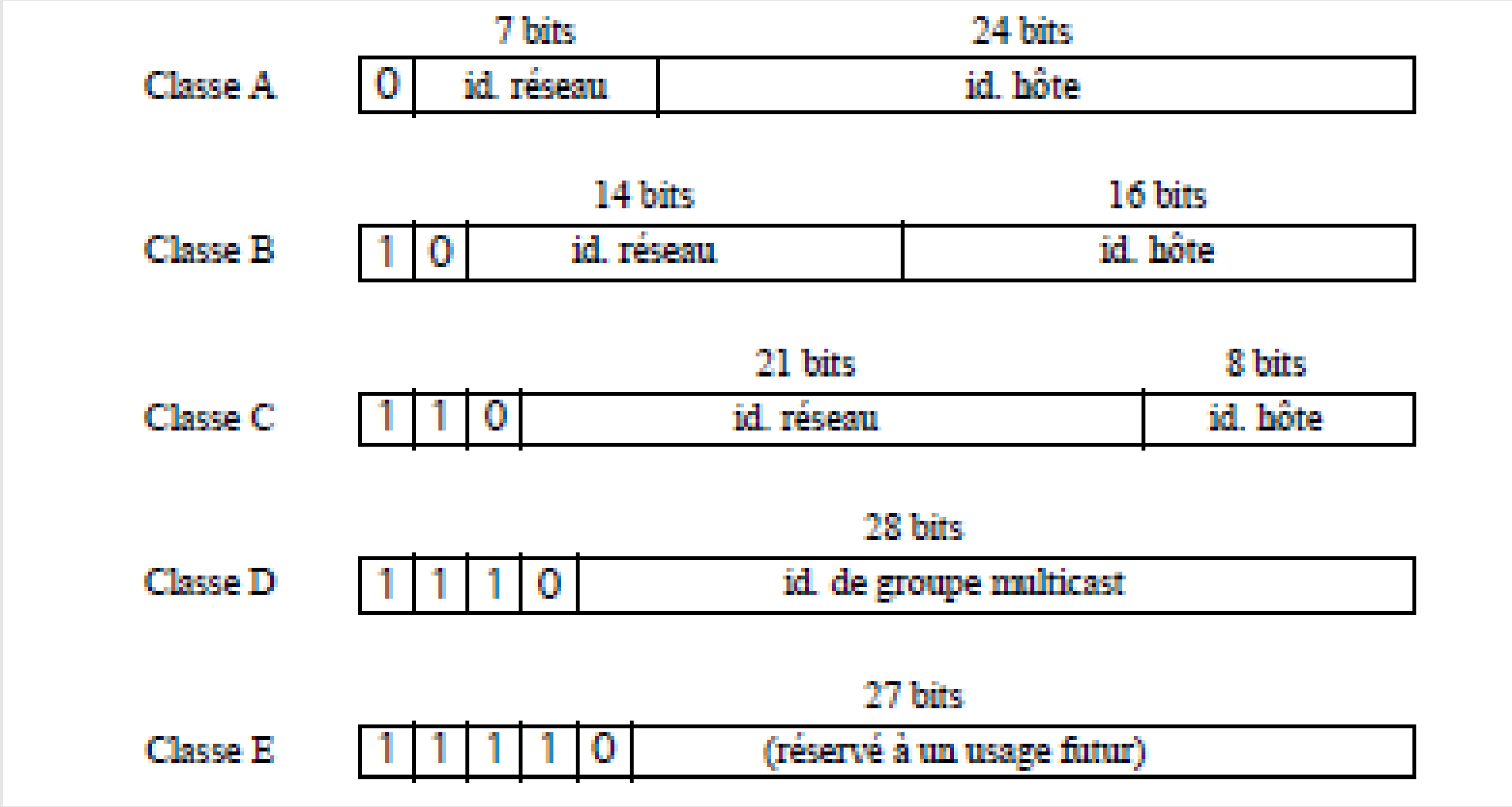
Introduction



Adresses IP

- Une machine (appelée aussi hôte ou host) est identifiée dans l'Internet par son adresse. L'adresse IP d'une machine correspond à un numéro qui est unique dans le monde.
- L'adresse utilisée par la version actuelle du protocole IP (adresse IP), comporte deux champs: le champ adresse réseau (Network) dans l'Internet et le champ adresse hôte (Host) dans le réseau. Sa taille est de 4 octets (32 bits). Elle est souvent donnée en notation décimale pointée (ex: 127.95.35.54).
- Il existe actuellement cinq classes d'adresses IP. Les trois premières permettent de gérer des réseaux de tailles diverses. La classe D permet de gérer une communication multipoint (un message est envoyé à plusieurs machines à la fois). La classe E est réservée pour la version d'IP IPv6 qui stockera les adresses IP dans 16 octets.

Adresses IP


7 bits
24 bits
Classe A
0
id. réseau
id. hôte
14 bits
16 bits
Classe B
1
0
id. réseau
id. hôte
21 bits
8 bits
Classe C
1
1
0
id. réseau
id. hôte
28 bits
Classe D
1
1
1
0
id. de groupe multicast
27 bits
Classe E
1
1
1
1
0
(réservé à un usage futur)

La classe InetAddress

- La classe java.net.InetAddress permet de représenter les adresses IP. Chaque objet de cette classe possède deux champs *hostName et address*
- **public static InetAddress getByName (String nom_hote)**

  Cette méthode utilise le DNS pour renvoyer une instance de la classe InetAddress représentant l'adresse Internet de la machine de nom nom_hote.
- **public static InetAddress getLocalHost ()**

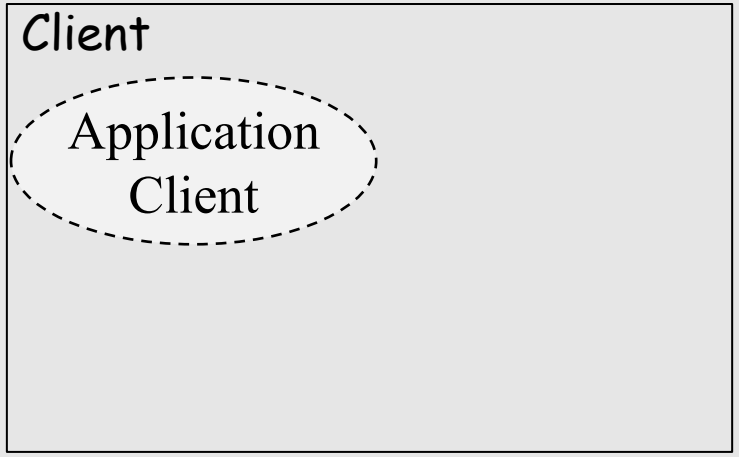
  Renvoie une instance de la classe InetAddress représentant l'adresse Internet de la machine locale

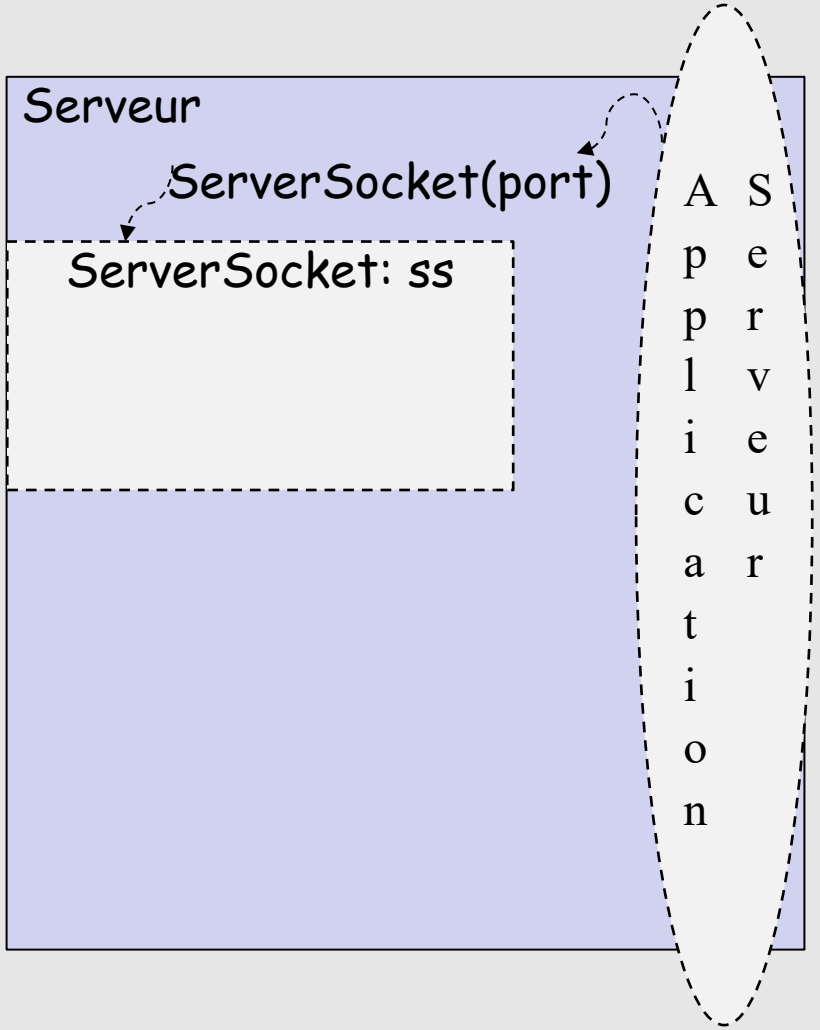
Les sockets

- Les sockets sont les terminaisons des connections entre machines hôtes et peuvent être utilisées pour envoyer et recevoir des données.
- Il y a deux types de sockets: *socket serveur* et socket client.
- *Socket serveur pour* recevoir les reqûetes des clients.
- Socket *client* peut être utilisé pour l'envoi et la réception des données.
- Le protocole TCP offre un service en mode connecté et fiable. Les données sont délivrées dans l'ordre de leur émission.

II-1 Procédure de connexion avec Java

Le serveur commence par créer une ServerSocket pour recevoir les demandes de connexion

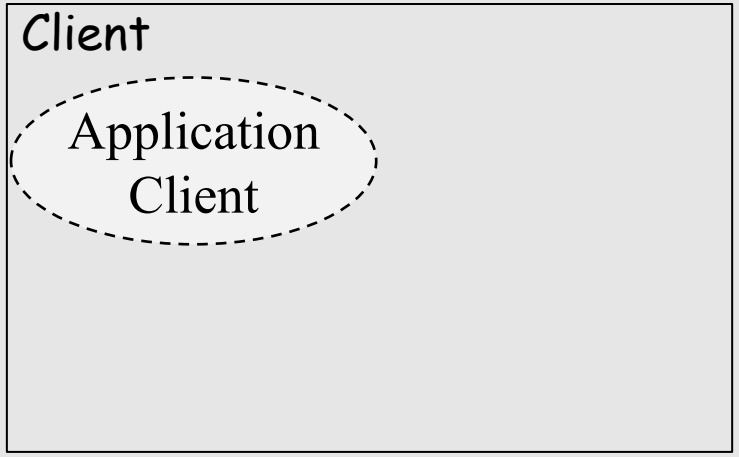


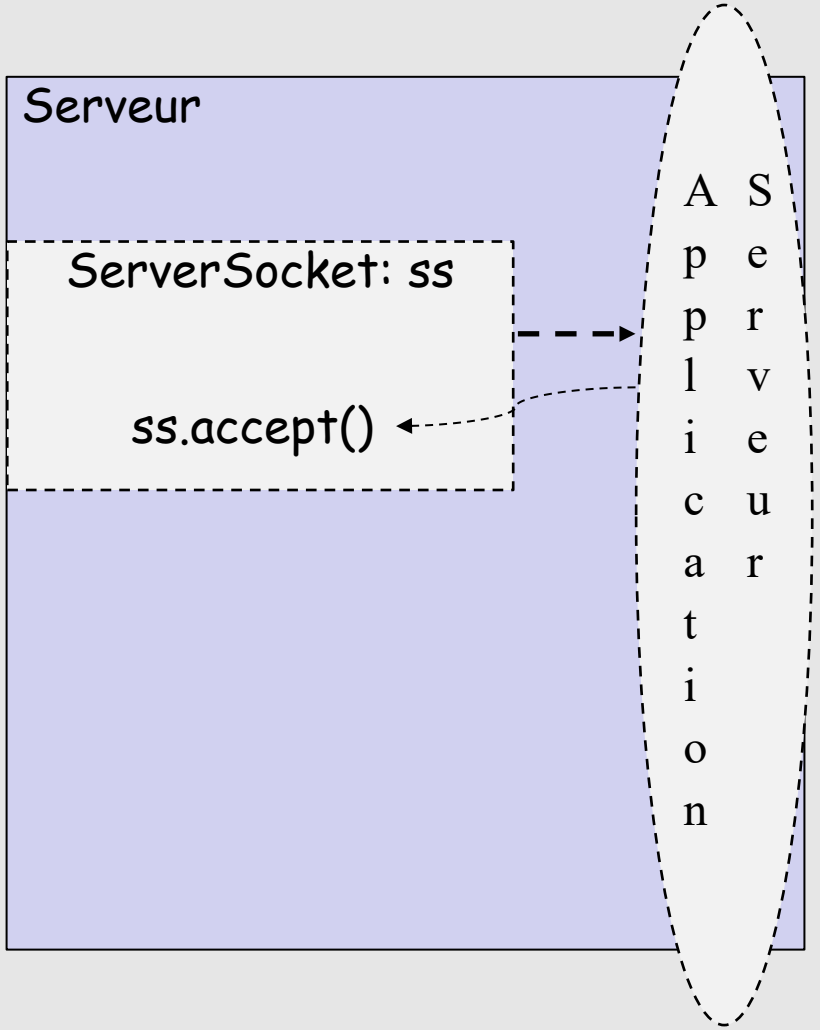


Procédure avec Java

II-1 Procédure de connexion avec Java

Le serveur se met en attente des demandes de connexion

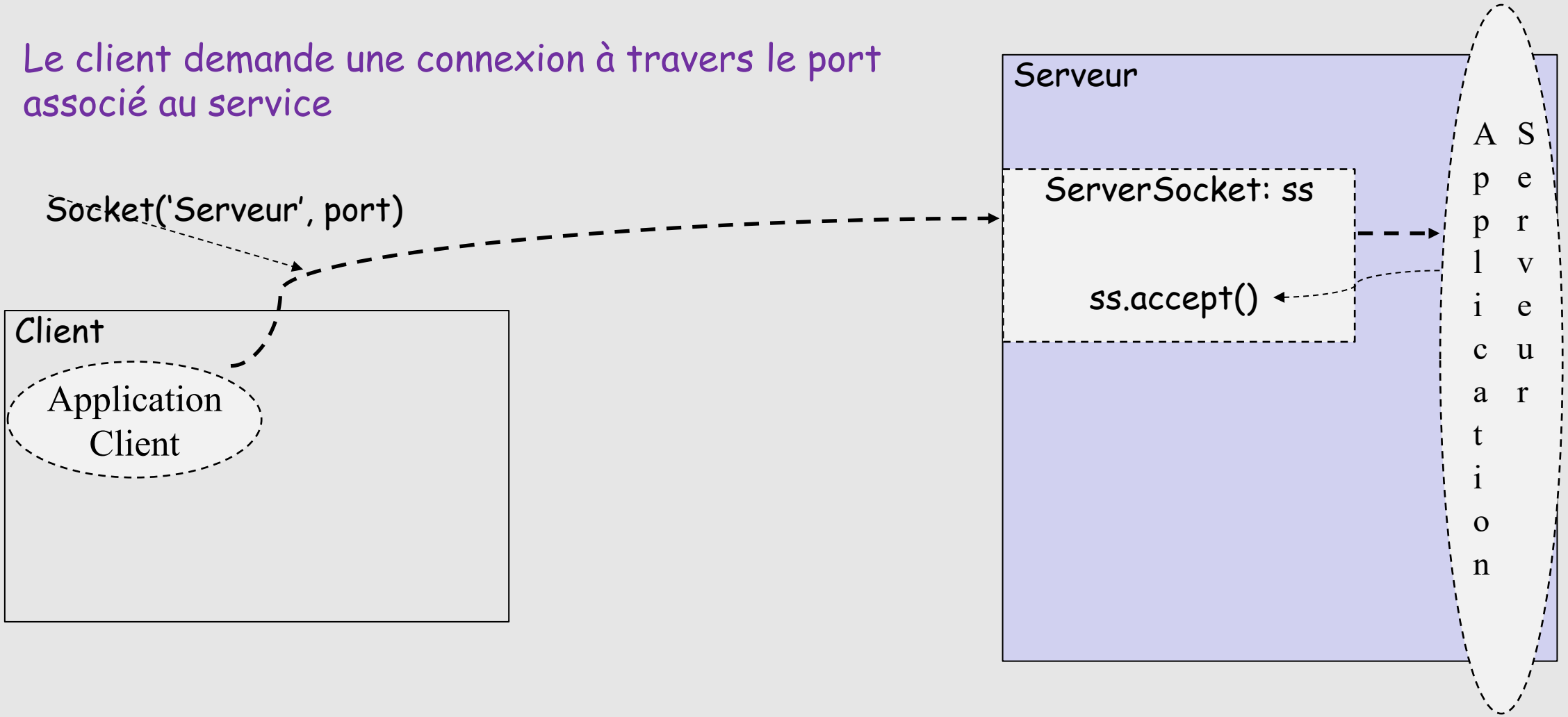




Procédure avec Java

II-1 Procédure de connexion avec Java

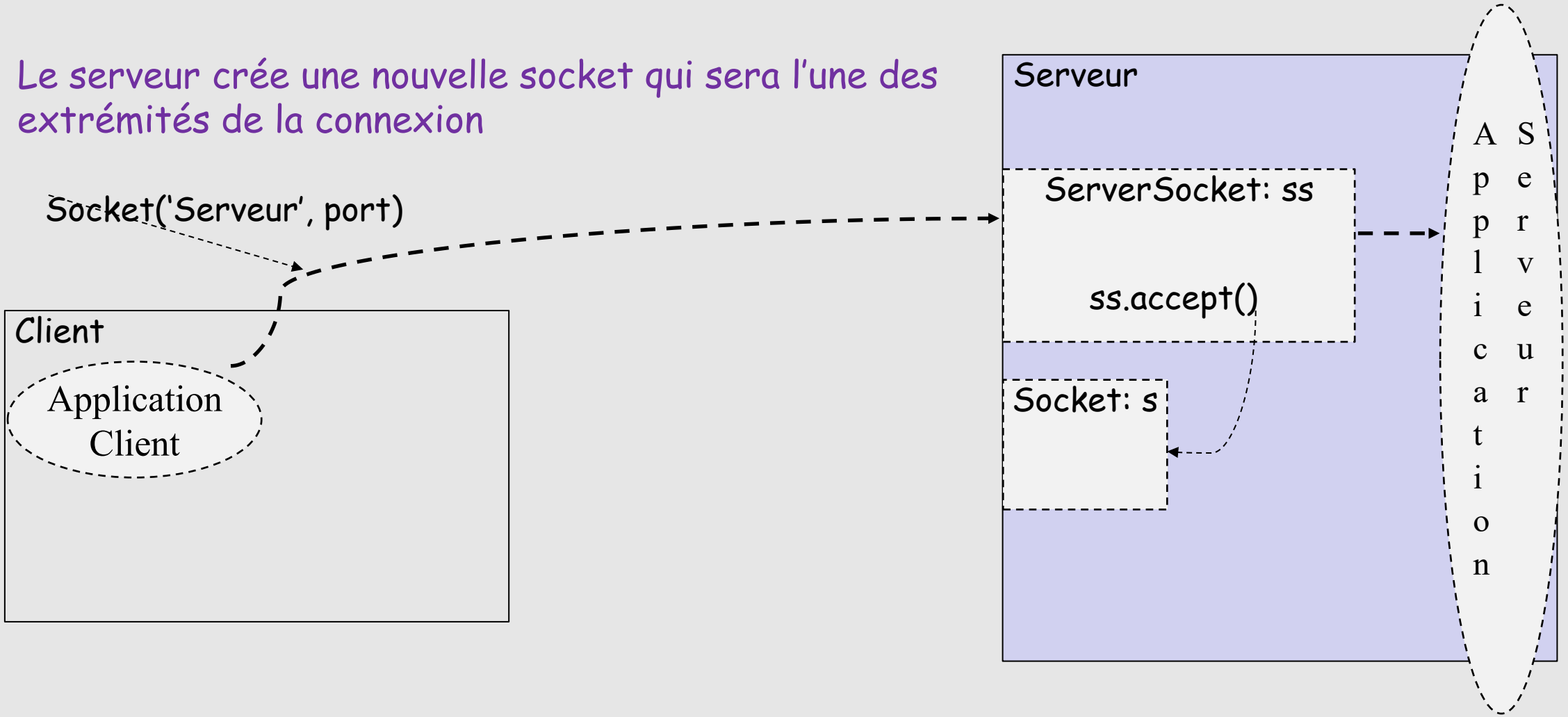
Le client demande une connexion à travers le port associé au service



Procédure avec Java

II-1 Procédure de connexion avec Java

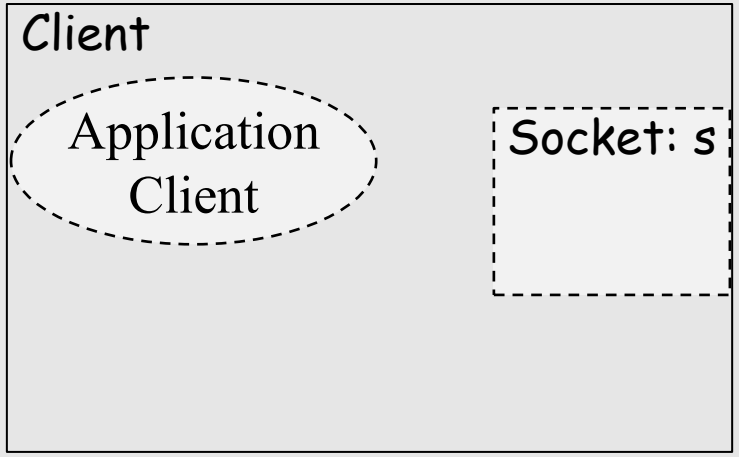
Le serveur crée une nouvelle socket qui sera l'une des extrémités de la connexion

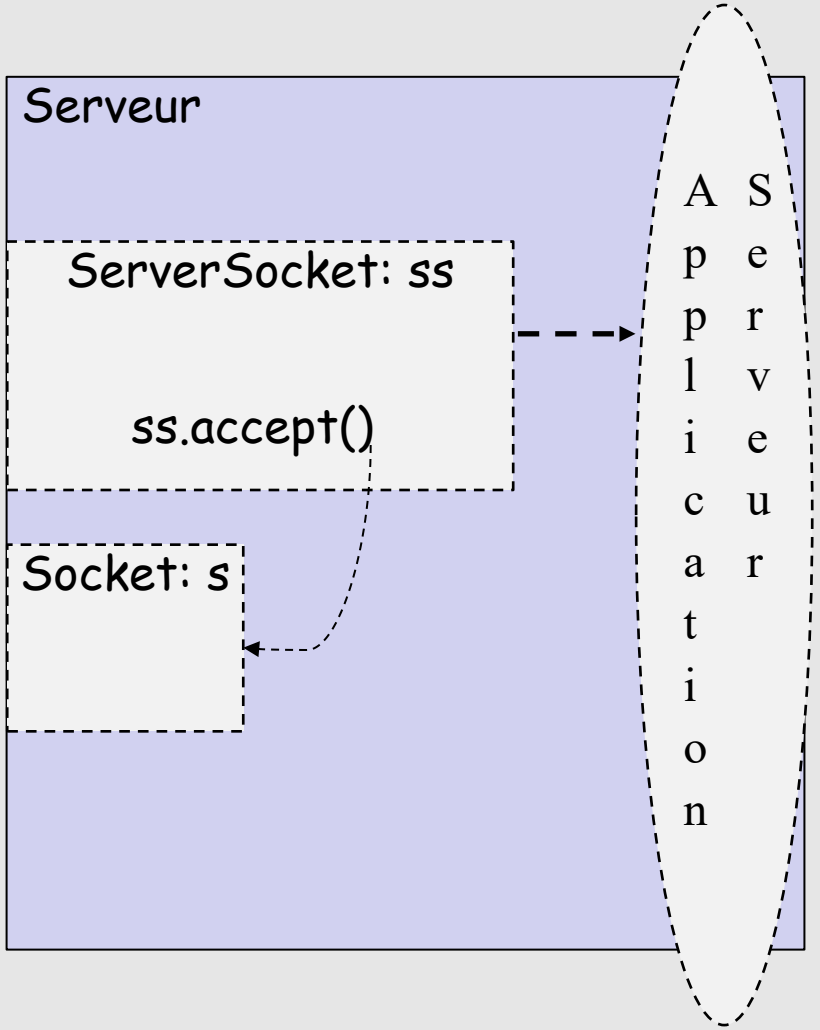


Procédure avec Java

II-1 Procédure de connexion avec Java

Le client obtient la socket qui est l'extrémité de la connexion côté client

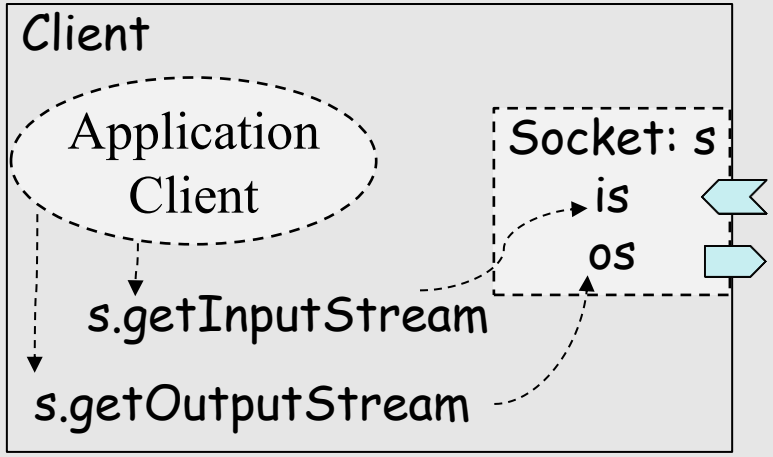


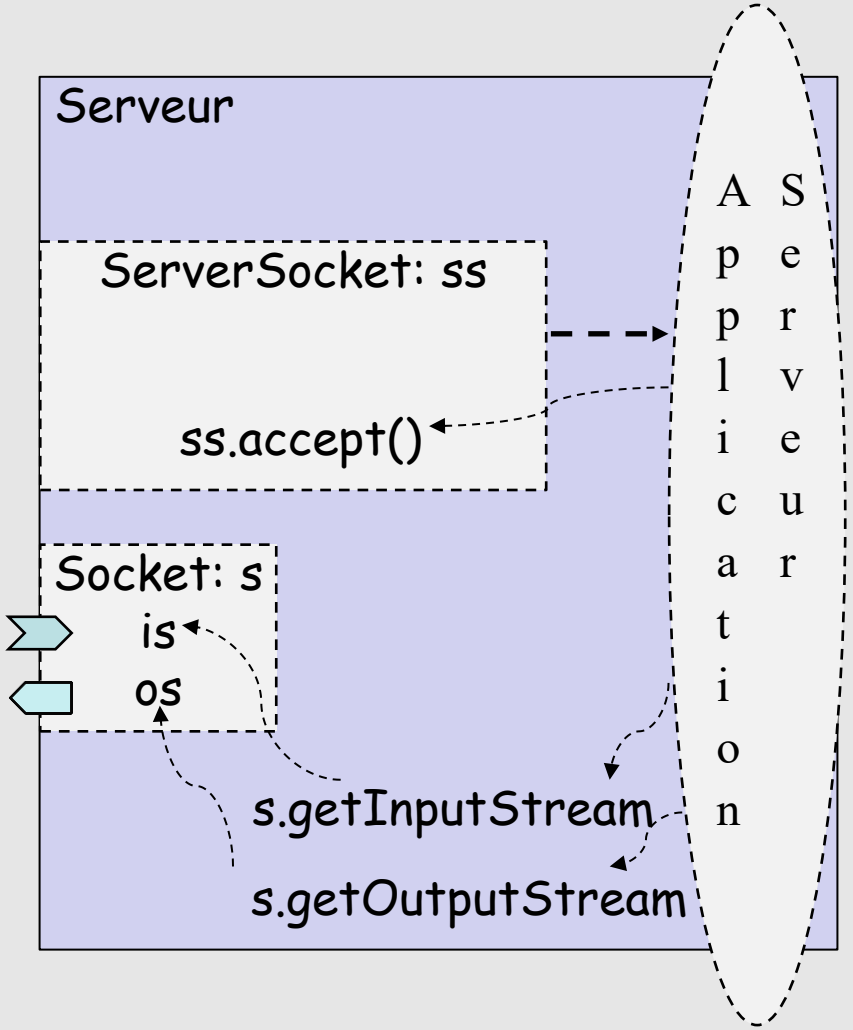


Procédure avec Java

II-1 Procédure de connexion avec Java

Le client et le serveur créent les InputStream et OutputStream associés aux deux sockets

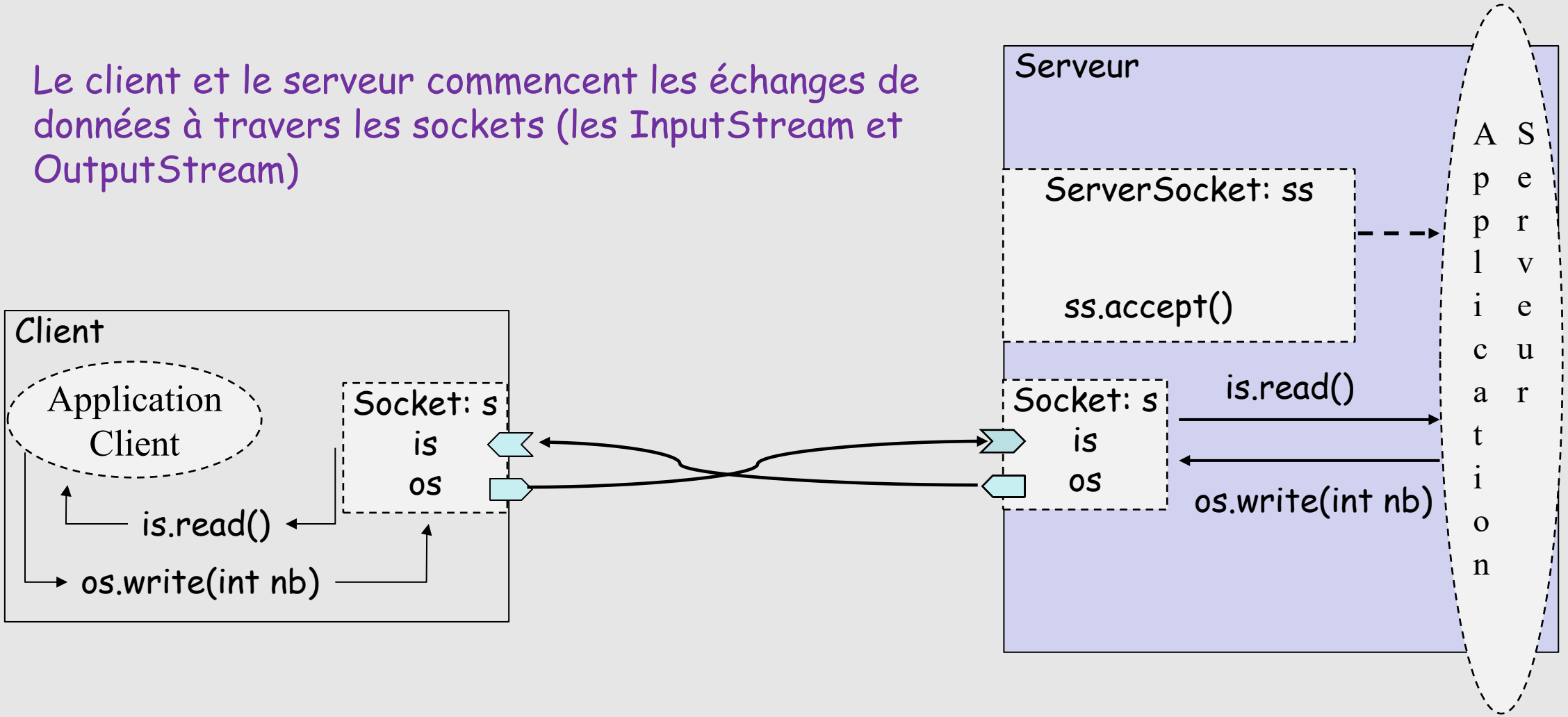




Procédure avec Java

II-1 Procédure de connexion avec Java

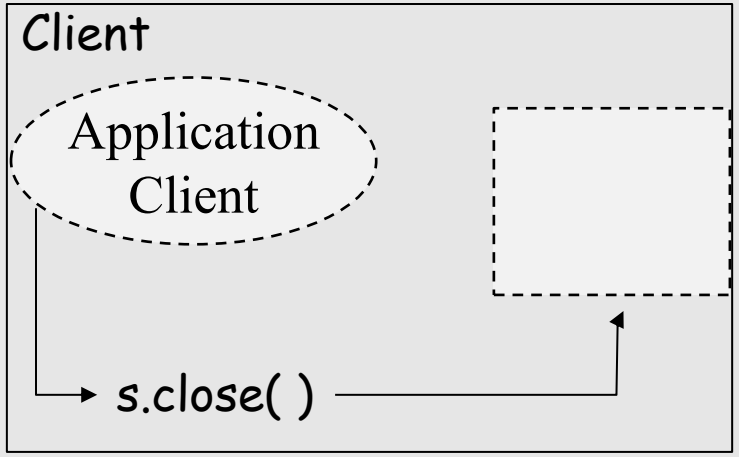
Le client et le serveur commencent les échanges de données à travers les sockets (les InputStream et OutputStream)

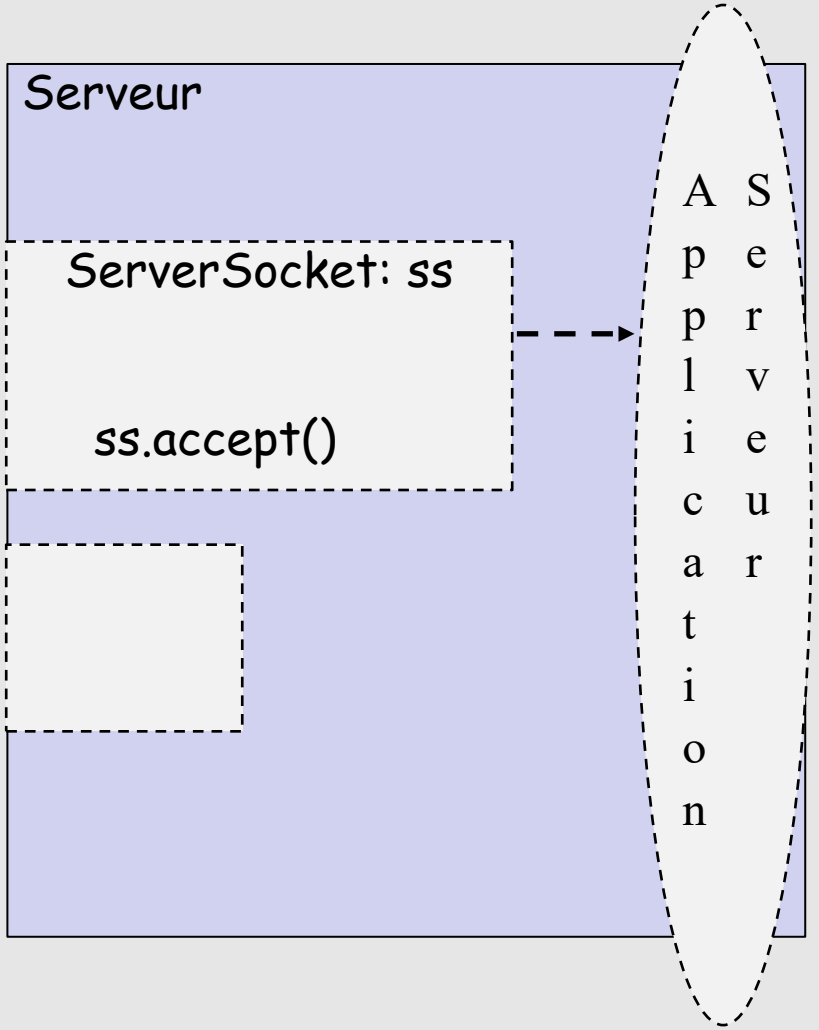


Procédure avec Java

II-1 Procédure de connexion avec Java

Le client et le serveur commencent les échanges de données à travers les sockets (les InputStream et OutputStream)





Procédure avec Java

II-3 Les Sockets Serveur (Java)

- Une socket serveur est une instance de la classe ServerSocket et peut être créee par l'un des ces constructeurs:

  ServerSocket(int port)

  ServerSocket(int port, int backlog)

- port: numéro de port sur lequel le serveur se mettra à l'écoute de requêtes de la part des clients.
- backlog: longeur maximum de la file des clients en attente d'être servis (50 par default).

II-3.1 Les méthodes des Sockets Serveur(Java)

- Socket accept()

  Attend une requête de connexion. Le thread qui exécute la méthode sera bloqué jusqu'à ce qu'une requête soit reçue, à ce moment la méthode retourne une socket client.

  Il y a deux faits importants à noter à propos de la méthode accept(). Le premier est que cette méthode est bloquante. Le deuxième est que cette méthode crée et retourne une instance de Socket.

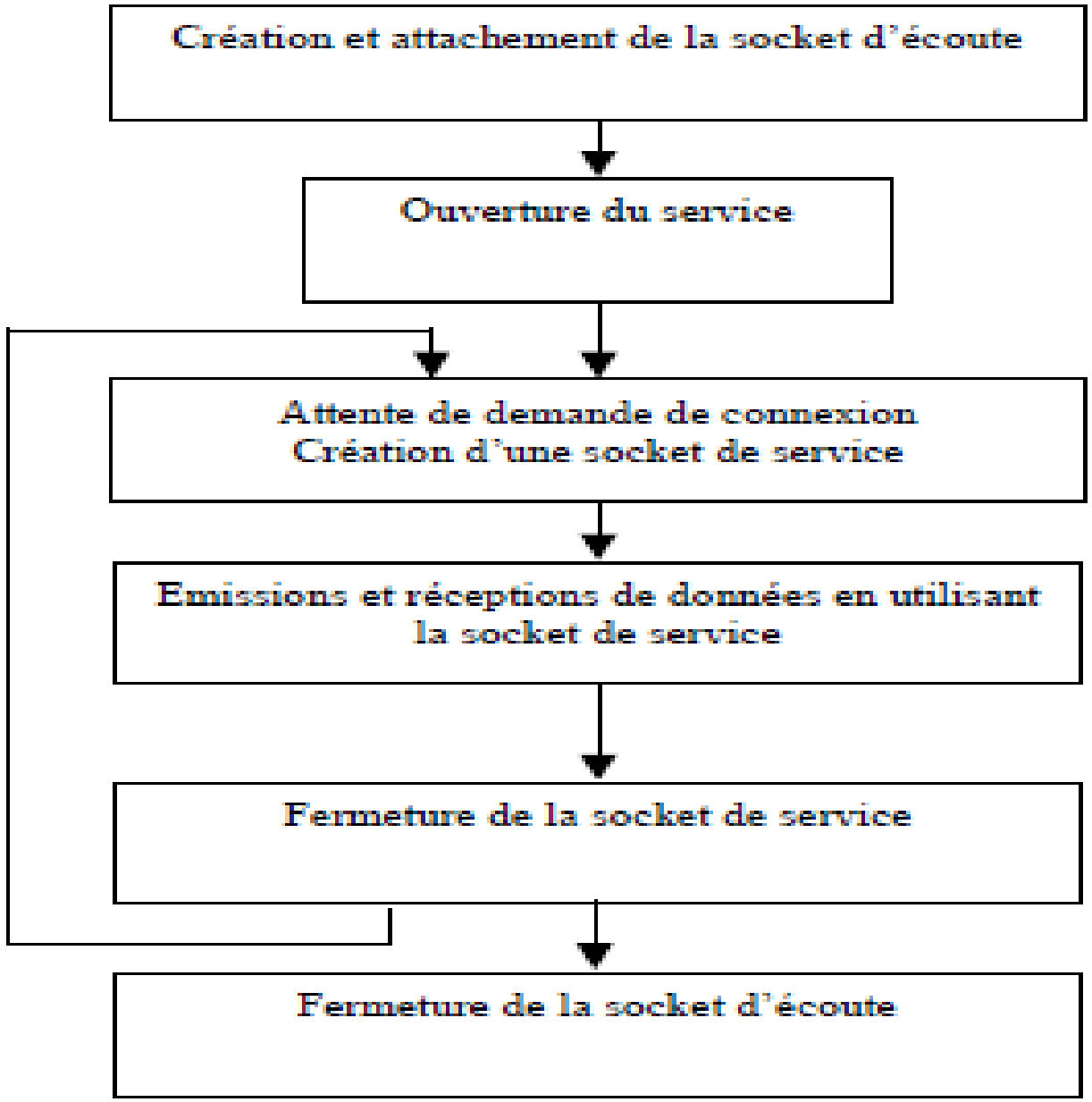
- void close()

  Arrête de recevoir des requêtes des clients.!

II-3.2 Usage typique de ServerSocket(Java)

- try {

```
        ServerSocket s = new ServerSocket(port);
    while (true) {
            Socket incoming = s.accept();
            «Traite un client»
            incoming.close();
    }
    s.close();
    } catch (IOException e) {
        «Traite l'exception» }
```



Fonctionnement du serveur

II-4 Les Sockets Client(Java)

- Une socket client est une instance de la classe Socket et peut être obtenue de deux manières:

(1) Côté serveur, comme valeur retournée par la méthode accept()

(2) Côté client en utilisant le constructeur

Socket(String host, int port)

host: l'adresse de la machine hôte

port: le numéro de port

II-4.1 Les méthodes des Sockets Client(Java)

- getInputStream()

  Retourne un objet InputStream pour recevoire les données(bytes)

- getOutputStream()

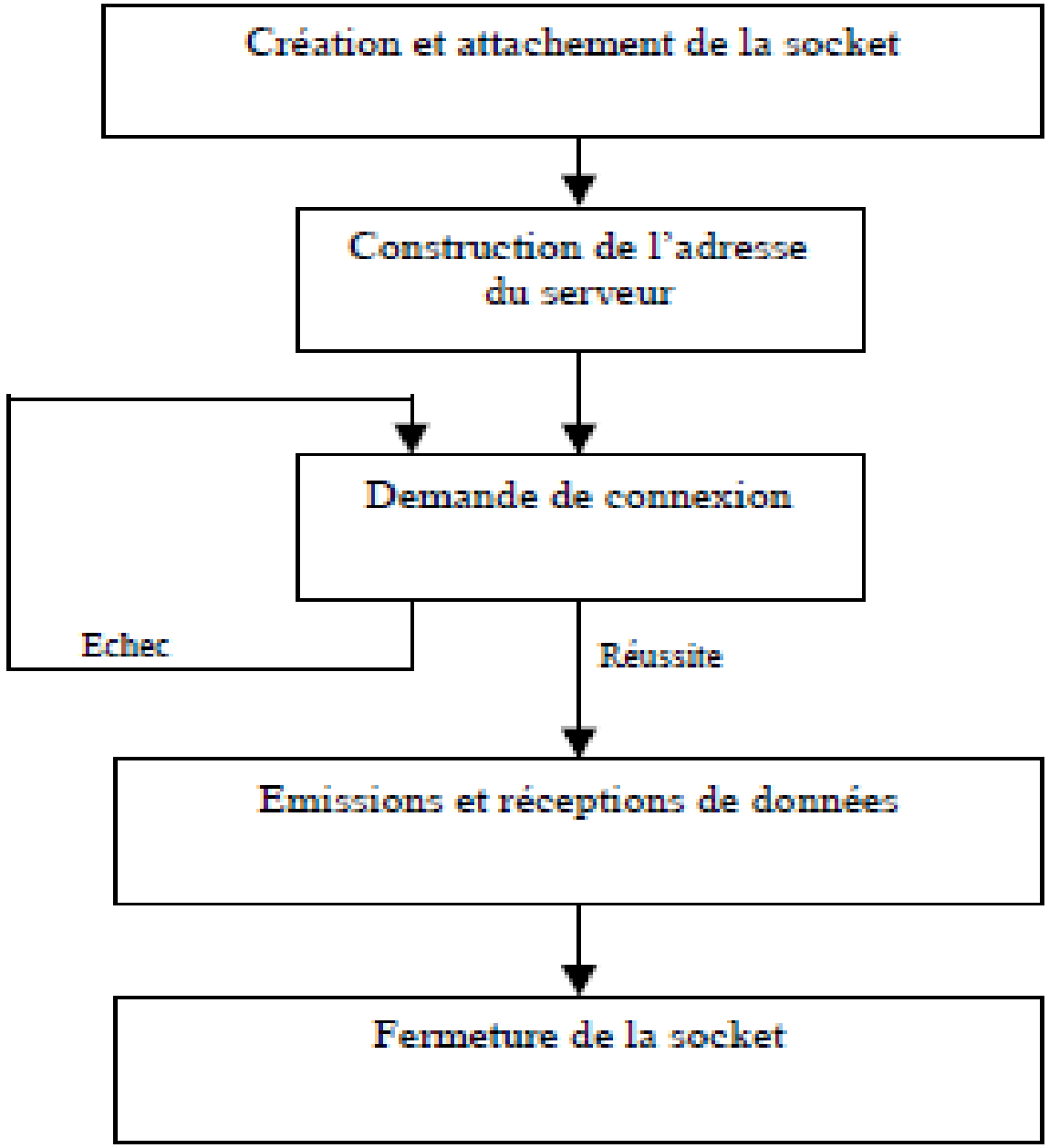
  Retourns un objet OutputStream pour envoyer des données(bytes)

- close()

  Ferme la socket(et par voie de conséquence la connexion)

II-4.2 Usage typique de Socket(Java)

```
try {
        Socket socket = new Socket(host, port);
        BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(
        new OutputStreamWriter(socket.getOutputStream()));
        «Envoie et reçoit les données»
        in.close();
        out.close(); socket.close();} catch (IOException e) {
        «Traite l'exception» }
```



Fonctionnement du client

Résumé sur les lectures / écritures

- InputStream : pour lire des octets
- InputStreamReader : pour lire des caractères
- BufferedReader : pour lire des caractères bufferisés (permet de lire des lignes)
- OutputStream : pour écrire des octets
- OutputStreamWriter : pour écrire des caractères
- PrintWriter : pour écrire tout type de données

Traiter correctement les exceptions

- La signature de la création de socket est :

public Socket(String host, int port) throws UnknownHostException, IOException

- En java, il est important de catcher les exceptions afin d'éviter qu'un programme crashe brutalement
- En programmation réseaux, c'est d'autant plus important car les problèmes peuvent être multiples :
  - mauvais numéro de port, hôte inconnu, problème de lecture ou d'écriture sur un flux

Méthodes informatives

- public InetAddress getInetAddress ()
- public int getPort ()
- Ces méthodes renvoient l'adresse Internet et le port distants auquel le socket est connecté.
- public InetAddress getLocalAddress ()
- public int getLocalPort ()
- Ces méthodes renvoient l'adresse Internet et le port locaux que le socket utilise.